

FFR105_HP2_2026

Erik Albinsson
2003/06/11

September 2026

Problem 2.1

In this problem the Ant System algorithm was implemented according to Algorithm 4.1 in [1] and was used to search for a short Traveling Salesman Problem tour over the 50 given cities. The shortest path found had a total length of 99.12. The corresponding path is shown in figure 1. The tour visits each of the 50 cities exactly once and returns to the starting city.

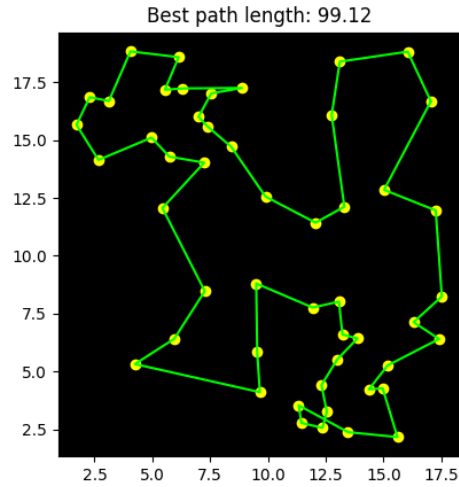


Figure 1: The best found path to the problem found by the Ant System algorithm of total length 99.12.

Problem 2.2

The function

$$f(x_1, x_2) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2$$

was investigated over the domain $(x_1, x_2) \in [-5, 5]$. A contour plot of $\log(0.01 + f(x_1, x_2))$, seen in figure 2, was used to visually identify and affirm the minima found by the implemented particle swarm optimization (PSO) algorithm.

The PSO algorithm was implemented in accordance with Algorithm 5.1 in [1] (also including the inertia weight parameter) and was run 50 times using different randomly sampled initial particle positions. Four distinct local minima were found by the algorithm. Their locations and corresponding function values are shown in table 1.

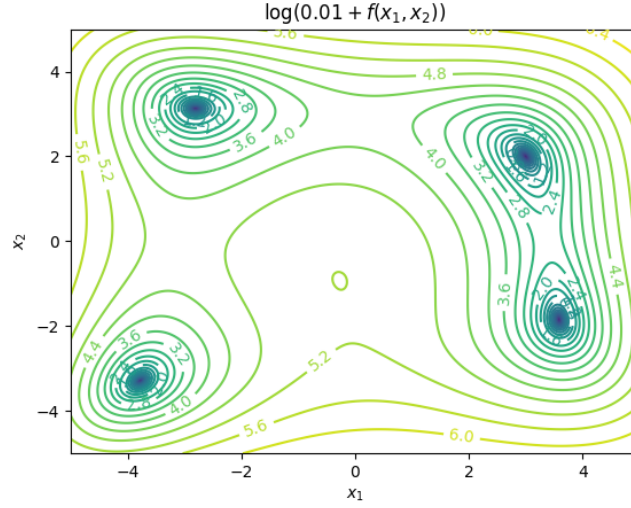


Figure 2: Contour plot of $\log(0.01 + f(x_1, x_2))$ over $(x_1, x_2) \in [-5, 5]$ with four minima visually identifiable.

x_1	x_2	$f(x_1, x_2)$
3.000000	2.000000	8.72×10^{-28}
3.584428	-1.848127	1.58×10^{-29}
-2.805118	3.131313	7.10×10^{-30}
-3.779310	-3.283186	4.37×10^{-27}

Table 1: Local minima of the function found using particle swarm optimization (PSO).

Problem 2.3

For this problem a genetic algorithm (GA) was implemented to optimize the weights for a feedforward neural network, which was used for controlling the braking system of a truck. The network has the 3 inputs $\frac{v}{v_{max}}$, $\frac{\alpha}{\alpha_{max}}$ and $\frac{T_b}{T_{max}}$,

one hidden layer with 5 neurons and 2 outputs representing the brake pedal pressure P_p and the desired gear change (truck's driver actions). The logistic sigmoid function, defined as

$$\sigma(s) = \frac{1}{1 + e^{-cs}},$$

was used as activation function with parameter $c = 2$. The chromosome in the GA contains all network weights, including the bias weights and encoding and decoding functions were implemented to alternate between the two. Weights were encoded as values between 0 and 1 and decoded to the interval $[-1, 1]$ when constructing the neural network.

The truck model was simulated using a first-order discretization with $\delta t = 0.1s$. The initial conditions were $x = 0m$, $v = 20m/s$, $T_b = 500K$ and gear 7. At each time step the neural network determined the brake pressure and desired gear change, where gear changes were limited to one every two seconds for realism. The simulation terminated if the truck either exceeded v_{max} , fell below v_{min} , exceeded T_{max} (failure) or reached the end of the 1000m slope (success).

In order to develop and test the neural network, 10 training slopes, 5 validation slopes and 5 test slopes were implemented with varying slope angles $\alpha(x)$, while satisfying the required criteria $0 < \alpha(x) < 10$. In order to evaluate the performance of a simulated truck on a slope the fitness function $F_i = \bar{v}_i d_i$ was used, where $\bar{v}_i$ is the average speed over the evaluated and d_i is the distance traveled before termination. As such with this definition of a fitness function, a higher fitness score is considered better. The fitness over a set of slopes was calculated as the average fitness over the slopes.

The GA used a population of 50 individuals where tournament selection, single-point crossover, mutation and elitism were used. The training fitness was used to guide the evolution, while the validation fitness was evaluated separately after each generation and was used to select the optimized network weights to store and to stop the optimization after the best found validation fitness had failed to improve over 30 consecutive generations.

Several independent GA runs were performed, where the maximum training and validation fitness as a function of the generation for the run selected in the final result is shown in figure 3. Here we see that the best network, based on validation fitness, was achieved in generation 51 in this network, with a training fitness of 18064 and validation fitness of 20497. The corresponding chromosome used was stored in *best_chromosome.py*.

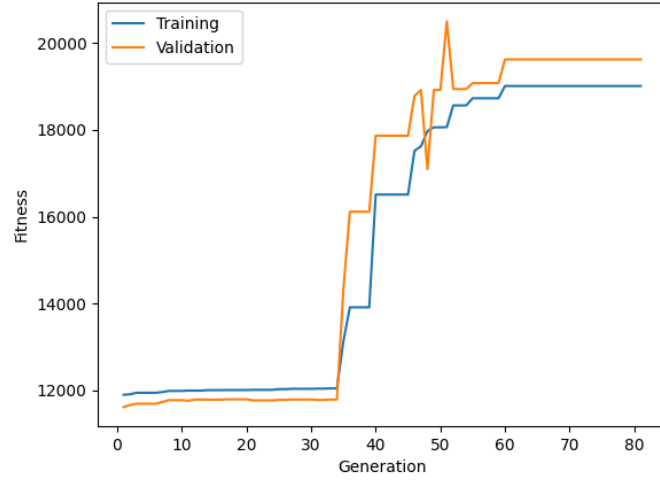


Figure 3: Maximum training and validation fitness vs. generation for the best GA run. The network with highest validation fitness was obtained in generation 51.

Applying the best found chromosome to the first test slope and look at the evolution of the brake temperature, speed, gear, brake pressure and slope angle as the truck traverses the slope, we get the results in figure 4.

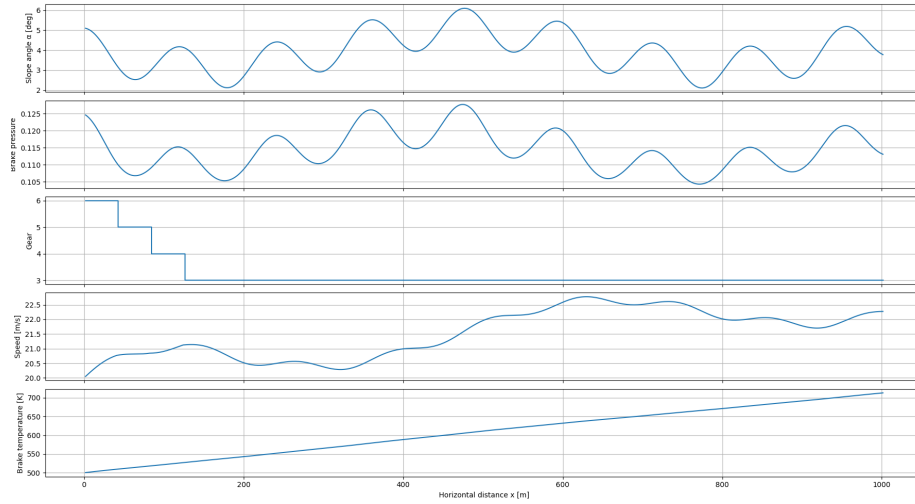


Figure 4: Brake temperature, speed, gear, brake pressure and slope angle vs. horizontal distance on the first test slope using the best found network.

Overall, the training and validation fitness both increased substantially over

the course of the first 50 generations in the form of rapid spurts from genetic changes in the chromosomes. Early stopping after 30 generations without validation improvement prevented overfitting to the training slopes and the selected network successfully completed the test slope while keeping speed, brake temperature and gear changes within the allowed limits. Thus indicating that the chosen fitness function $F_i = \bar{v}_i b_i$ guided the GA to a brake system which balanced high average speed while covering the full 1000m and not violating constraints.

Problem 2.4

Linear Genetic Programming (LGP) was used to approximate the unknown function from the supplied data in the form of

$$g(x) = \frac{a_0 + a_1x + a_2x^2 + \cdots + a_px^p}{b_0 + b_1x + b_2x^2 + \cdots + b_qx^q}.$$

Each chromosome consists of instructions containing four genes, those being an operator, a destination register and 2 operand registers. The operators used were addition, subtraction, multiplication and division. Specific to this solution 3 variable registers and 3 constant registers were used, with the constant register containing 1, 2 and -1 .

For each data point, the first variable register was initialized with the input x , while the other variable registers were initialized to 0. The chromosome was then executed instruction by instruction. The final value in the first variable register was used as the predicted output of the function.

The error of a chromosome was measured as

$$e = \sqrt{\frac{1}{K} \sum_{k=1}^K (\hat{y}_k - y_k)^2}$$

and fitness was computed as $f = \frac{1}{e}$, meaning chromosomes with smaller errors receive higher fitness.

In the implementation, tournament selection was used with size of 5 and probability 0.75 and two-point crossover was used to combine chromosomes, with crossover probability of 0.8. An adaptive mutation probability was also used in order to help avoid stagnation. The base gene mutation had a probability of 0.04 per gene and when the best solution did not improve for a number of generations, the mutation probability was increased. When a improved solution was found, the mutation probability was reset to its base value. Instruction insertion and deletion was also included as additional mutation operations. Additionally, the chromosome length was restricted to a maximum of 400 genes, corresponding to 100 instructions. Elitism was also used in order to preserve properties from the best found gene.

The best found chromosome found by the LGP algorithm was the function

$$\frac{1 + 2x}{1 + 2x + x^2},$$

which had an root mean square error of approximately 0.23935. The comparison between the supplied data and the function produced by the LGP algorithm is shown in figure 5.

The result shows that the LGP algorithm was able to produce a function that approximates the supplied data with an error below 0.30.

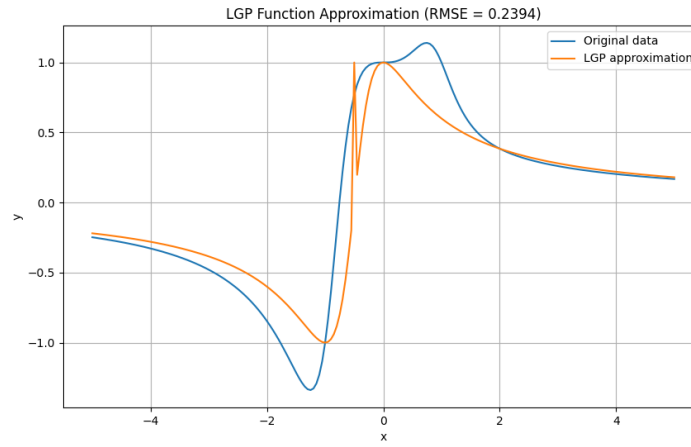


Figure 5: The supplied function data together with the function obtained using the best LGP chromosome.

References

- [1] M Wahde. *Biologically Inspired Optimization Methods: An Introduction*. WIT Press, 2008.